



Initiation aux réseaux d'entreprise

Partie 2/2

Ressource R101

1. Masque de sous-réseaux

Le masque est un séparateur entre la partie réseau et la partie machine d'une adresse IP (& logique).

Exemple : adresse 192.168.1.2 et masque 255.255.255.0

L'utilisation et la maîtrise des masques doivent pouvoir vous permettre d'une part, de savoir ce que vous manipulez, et d'autre part d'optimiser le fonctionnement de votre réseau.

L'utilisation des masques vous permettra de **segmenter** de la façon la plus correcte l'adressage de votre réseau, et ainsi de séparer les machines sensibles du reste du réseau, limiter les congestions, et prévoir l'évolution de votre réseau, etc...

La séparation d'un réseau en plusieurs sous-réseaux n'a pas que des avantages.

L'un des inconvénients majeurs est notamment la **complexification** des tables de routage étant donné le plus grand nombre de réseaux à router.

Le masque comme l'adresse IP est une suite de 4 octets, soit 32 bits (4 octets*8 bits).

Chacun de ces bits peut prendre la valeur 1 ou 0.

—→ Les bits à 1, du masque de sous-réseaux, représentent la partie réseau de l'adresse, et les bits à 0, la partie machine.

Ainsi, on fera une association entre une adresse IP et un masque pour savoir dans cette adresse IP, quelle est la partie réseau et quelle est la partie machine de l'adresse.

Le masque servant à faire la séparation en deux parties sur une adresse IP, il est donc indissociable de celle-ci.

Une adresse seule ne voudra rien dire puisqu'on ne saura pas quelle est la partie réseau et quelle est la partie machine.

De la même façon, un masque seul n'aura pas de valeur puisqu'on n'aura pas d'adresse sur laquelle l'appliquer.

	192	45	35	9
Adresse IP	11000000	00101101	00100011	00001001
	255	255	240	0
Masque	11111111	11111111	11110000	00000000
	192	45	32	0
Sous réseau	11000000	00101101	00100000	00000000
	0	0	3	9
Hôte	00000000	00000000	00000011	00001001

Masques de sous-réseau par défaut :

- Classe A : 255.0.0.0
- Classe B : 255.255.0.0
- Classe C : 255.255.255.0

En notation **CIDR**, on donne le nombre de bits (poids forts) à 1 donc le masque de classe A sera noté :

- @IP / 8, exemple 10.10.10.10/8
- La classe B est en / 16 et la classe C en / 24.

Exemple d'une configuration d'un ordinateur sous Windows avec la commande **ipconfig /all** (copie d'écran partielle) :

Remarque : la commande « **ipconfig /all** » sera entré sur l'invite de commande Windows (ou Linux)

Exemple : `C:\Users\pmatichard>ipconfig /all`

```
Nom de l'hôte . . . . . : EMPROA129151
Suffixe DNS principal . . . . . : ujmse.local
Type de noeud . . . . . : Hybride
Routage IP activé . . . . . : Non
Proxy WINS activé . . . . . : Non
Liste de recherche du suffixe DNS.: ujmse.local

Carte Ethernet Ethernet :

Statut du média. . . . . : Média déconnecté
Suffixe DNS propre à la connexion. . . :
Description. . . . . : Intel(R) Ethernet Connection (4) I219-LM
Adresse physique . . . . . : 8C-04-BA-05-49-D7
DHCP activé. . . . . : Oui
Configuration automatique activée. . . : Oui

Carte réseau sans fil Connexion au réseau local* 1 :

Statut du média. . . . . : Média déconnecté
Suffixe DNS propre à la connexion. . . :
Description. . . . . : Microsoft Wi-Fi Direct Virtual Adapter
Adresse physique . . . . . : 58-A0-23-93-34-70
DHCP activé. . . . . : Oui
Configuration automatique activée. . . : Oui

Carte réseau sans fil Connexion au réseau local* 10 :

Statut du média. . . . . : Média déconnecté
Suffixe DNS propre à la connexion. . . :
Description. . . . . : Microsoft Wi-Fi Direct Virtual Adapter #2
Adresse physique . . . . . : 5A-A0-23-93-34-6F
DHCP activé. . . . . : Oui
Configuration automatique activée. . . : Oui
```

- Exemple de configuration sous LINUX :

```
root@crpfs2:~# ifconfig
eth0      Link encap:Ethernet  HWaddr 08:00:27:fc:b5:25
          inet adr:192.168.10.1  Bcast:192.168.10.255  Masque:255.255.255.0
          adr inet6: fe80::a00:27ff:fe80:b525/64 Scope:Lien
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:4002 errors:0 dropped:0 overruns:0 frame:0
          TX packets:52 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 lg file transmission:1000
          RX bytes:363079 (354.5 KiB)  TX bytes:3612 (3.5 KiB)

lo        Link encap:Boucle locale
          inet adr:127.0.0.1  Masque:255.0.0.0
          adr inet6: ::1/128 Scope:Hôte
          UP LOOPBACK RUNNING  MTU:16436  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 lg file transmission:0
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
```

2. Protocoles et ports

Lorsqu'un datagramme est réceptionné et que son adresse destination correspond à l'adresse IP locale, il est transmis à la couche supérieure au protocole de transport correspondant au numéro contenu dans le champ **Protocol**.

0	4	8	15	16	24	31
Version	IHL	Type de service	Longueur Totale			
Identificateur			Flags	Positionnement fragment		
Durée de vie	Protocole		Cheksum de l'en-tête			
Adresse source						
Adresse destination						
Option				Bourrage		
Données						

Par exemple le numéro 6 correspond au protocole TCP tandis que 17 correspond à UDP.

Dès qu'IP a transmis les données entrantes au protocole de transport approprié, celui-ci les transmet au processus identifié par le **numéro de port**.

En effet les applications ou processus utilisant la pile **TCP/IP** sont identifiées au moyen de numéro de port, qui sont des valeurs de 16 bits.

Le numéro du port source et le numéro du port destination sont contenus dans les segments TCP et le paquet UDP.

Les numéros inférieurs à 256 sont réservés pour les "services réservés" et sont définis dans le RFC *Assigned Numbers*.

- Dans les systèmes UNIX, le fichier « */etc/services* » contient ces valeurs.
- Sous Windows, on le trouve sous
« *Windows/SYSTEM32/DRIVERS/ETC/services* »

La combinaison numéro de protocole et numéro de port permet d'identifier de manière unique un chemin de données.

Les **ports réservés** correspondent à des numéros de port standardisés permettant une connexion simplifiée puisque émetteur et récepteurs connaissent à l'avance les numéros à utiliser pour une opération donnée.

Par exemple tous les systèmes offrant le service **TELNET** (Telnet est un protocole informatique qui a été conçu pour interagir avec des ordinateurs distants), le proposent sur le **port 23**.

Il existe un deuxième type de numéro de port, appelé port alloué dynamiquement.

Ils sont utilisés par exemple pour le support de plusieurs utilisateurs.

Soit par exemple, 2 utilisateurs d'une même machine qui désirent faire un TELNET sur un même système.

Si le numéro standard 23 est attribué comme port source et destination à la première connexion, il ne peut pas être aussi utilisé pour la deuxième.

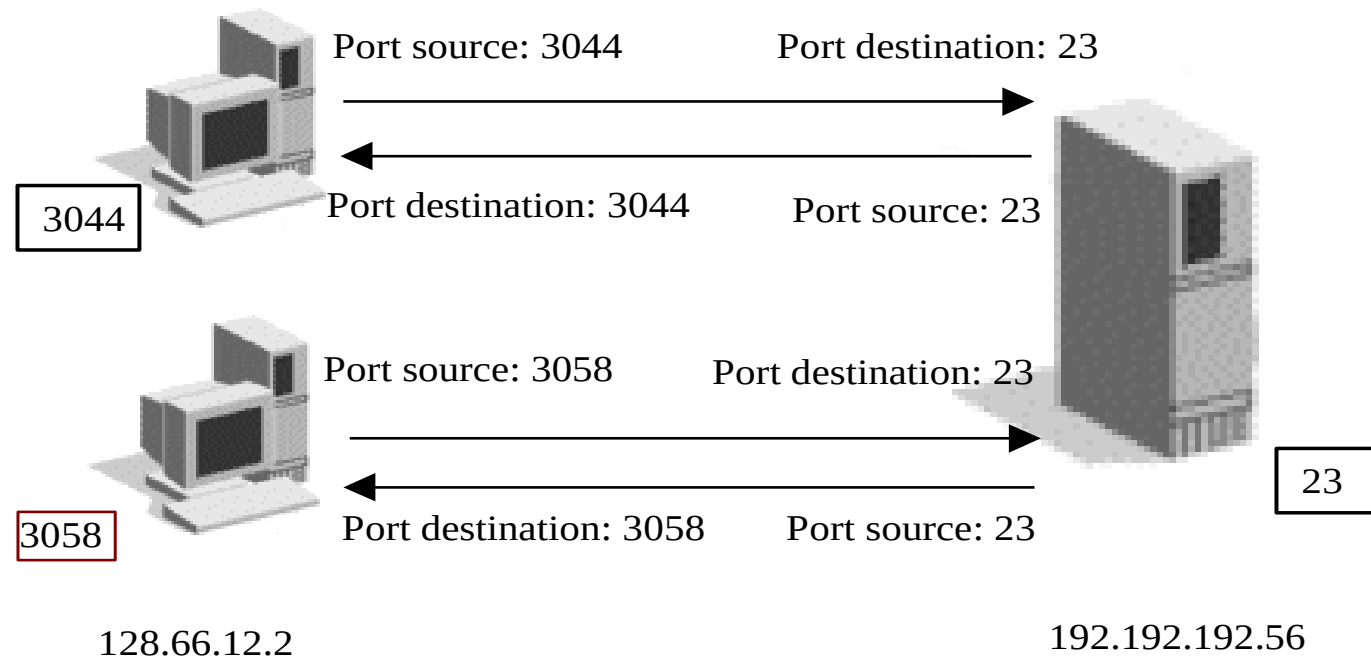
Pour pallier à ce problème, un numéro de port source est attribué dynamiquement tandis que le numéro réservé 23 est utilisé comme port destination.

Les deux utilisateurs vont donc avoir deux paires de numéro différentes.

Exemple

Processus source:
2 x Clients TELNET

Processus destination:
Serveur TELNET



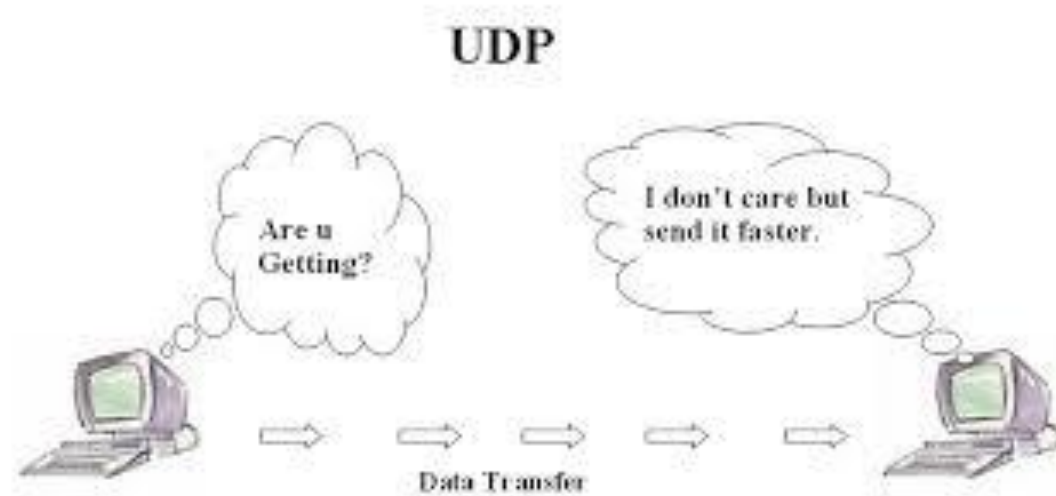
Le groupement adresse IP + N°port est appelé **socket**. Il identifie de manière unique le processus concerné.

Dans l'exemple, le premier client Telnet possède le **socket 128.66.12.2:3044** tandis que le serveur Telnet possède le **socket 192.192.192.56:23**

La connexion est définie par la paire de ces deux *sockets*.

2. Le protocole UDP

Le **protocole UDP** (rfc 768) utilise IP pour acheminer, d'un ordinateur à un autre, en **mode non fiable** des datagrammes qui lui sont transmis par une application.



- UDP n'utilise pas d'accusé de réception et ne peut donc pas garantir que les données ont bien été reçues.
- Il ne réordonne pas les messages si ceux-ci n'arrivent pas dans l'ordre dans lequel ils ont été émis et il n'assure pas non plus de contrôle de flux.

Il se peut donc que le récepteur ne soit pas apte à faire face au flux de datagrammes qui lui arrivent.

C'est donc à l'application qui utilise UDP de gérer les problèmes de perte de messages, duplications, retards, dé-séquencement, ...

Exemples d'utilisation :

- les protocoles DHCP, DNS, SNMP ;
- Le streaming ;
- les jeux en réseau (exemple : jeux de tir à la première personne).

Cependant, UDP fournit un service supplémentaire par rapport à IP, il permet de distinguer plusieurs applications destinataires sur la même machine par l'intermédiaire des ports.

Un port est une destination abstraite sur une machine identifiée par un numéro qui sert d'interface à l'application pour recevoir et émettre des données.

Par exemple :

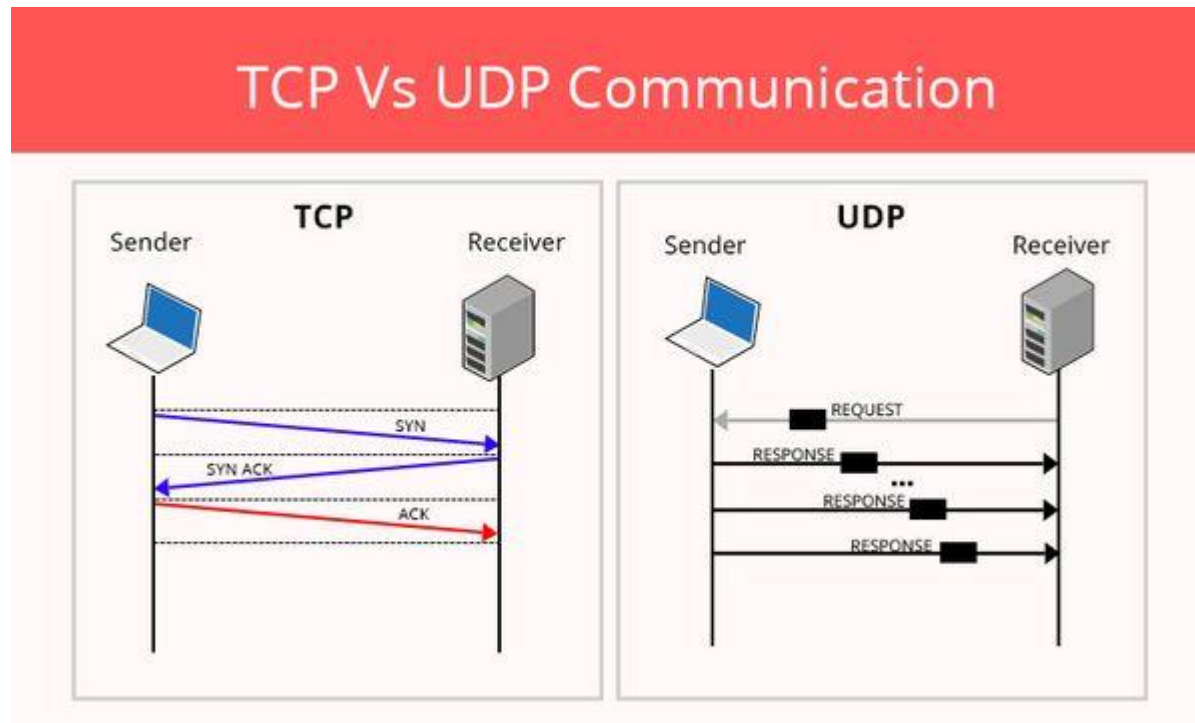
- l'application **TFTP** (Trivial File Transfert Protocol, un protocole simple de transfert de fichiers) utilise le port 69
- l'application **SNMP** (Simple Network Management Protocol, permet aux administrateurs réseau de gérer les équipements du réseau, de superviser et de diagnostiquer des problèmes réseaux) utilise le port 1612.

Chaque datagramme émis par UDP est encapsulé dans un datagramme IP en y fixant à 17 la valeur du protocole.

+	Bits 0 - 7	8 - 15	16 - 23	24 - 31
-96	Adresse Source			
-64	Adresse Destination			
-32	Zéros	Protocole	Taille UDP	
0	Port Source		Port Destination	
32	Longueur		Somme de contrôle	
64	Données			

3. Le protocole TCP

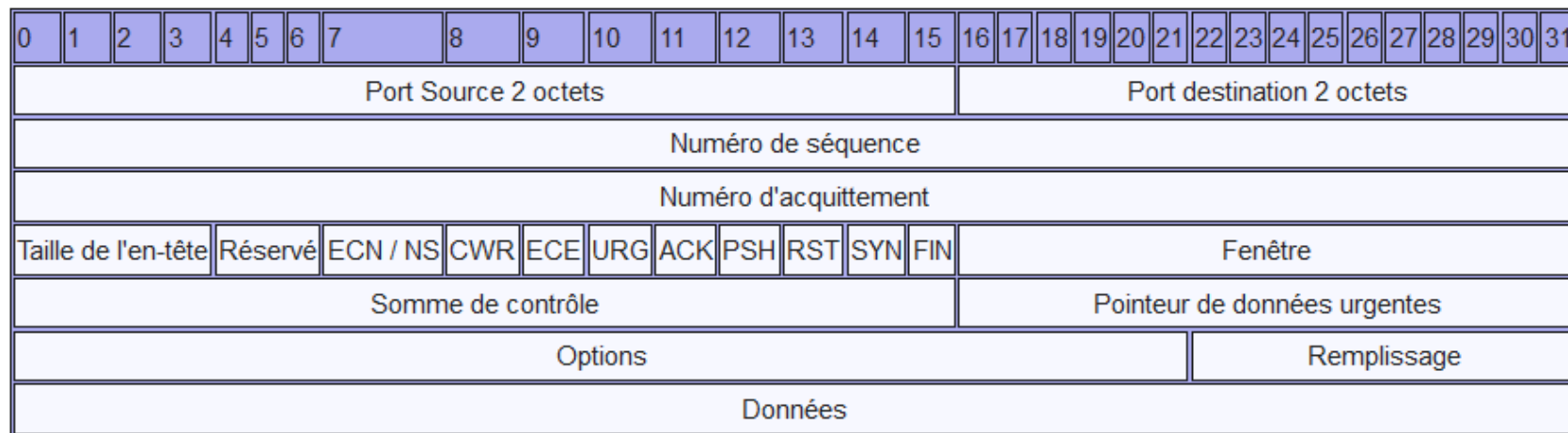
Contrairement à UDP, **TCP** est un protocole qui procure **un service de flux d'octets orienté connexion et fiable**.



Le terme orienté connexion signifie que les applications dialoguant à travers TCP sont considérées l'une comme un serveur, l'autre comme un client, et qu'elles doivent établir une connexion avant de pouvoir dialoguer (comme dans le cas de l'utilisation du téléphone).

Les ordinateurs vérifient donc préalablement que le transfert est autorisé, que les deux machines sont prêtes en s'échangeant des messages spécifiques.

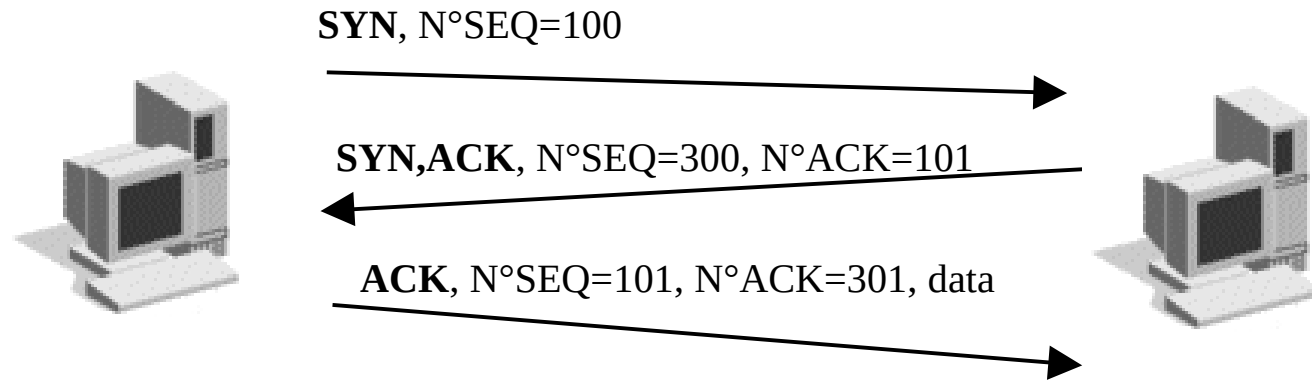
Datagramme TCP :



TCP utilise le mécanisme appelé **PAR** (*Positive Acknowledgment with retransmission*) :

- Côté émetteur, tout segment envoyé est réexpédié si aucun acquittement positif est reçu dans un laps de temps donné.
- Côté récepteur, chaque segment reçu est contrôlé par l'intermédiaire d'une somme de contrôle. Si le segment est correct, un acquittement positif est envoyé vers l'expéditeur, sinon il est simplement ignoré par le récepteur. Après le délai d'attente, l'expéditeur retransmettra le segment.

L'établissement de la connexion se fait par une poignée de main (SYN + ACK) comme sur l'exemple suivant:

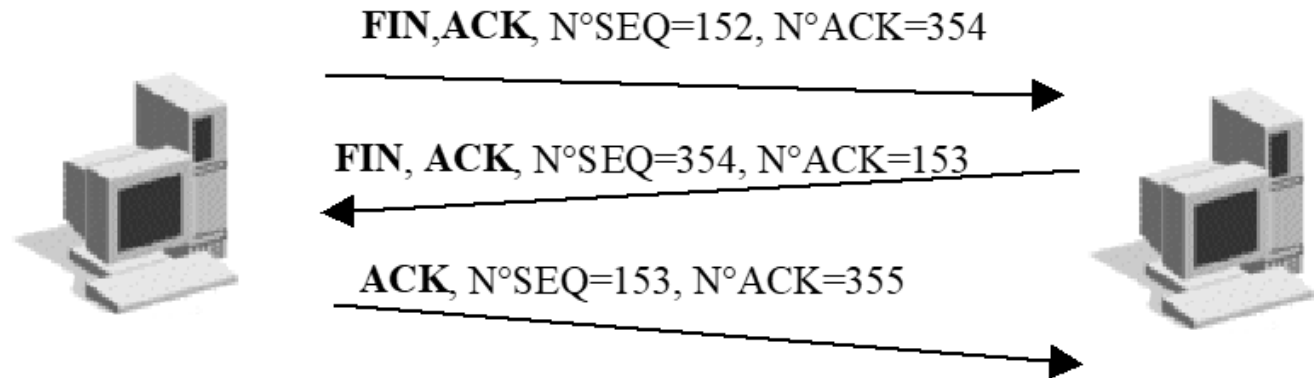


Les segments TCP utilisent deux compteurs, un **numéro de séquence** qui précise le numéro d'ordre du premier octet de données du segment.

Un **numéro d'acquittement** qui précise le prochain numéro de séquence attendu.

- A envoie un segment **SYN** (synchroniser les numéros de séquence utilisés dans l'établissement de connexion) vers B contenant un numéro dans le champ numéro de séquence, ce qui veut dire que A souhaite établir une connexion avec ce numéro comme numéro initial, ce numéro est appelé **ISN** (*Initial Sequence Number*).
- B renvoie un segment **SYN, ACK** (Acknowledge, Indicateur d'accusé de réception utilisé dans l'établissement de la connexion et la fin de la session) avec le numéro d'acquittement = ISN+1 et avec aussi un numéro de séquence initial puisque l'échange est bidirectionnel.
- Pour finir A renvoie un segment ACK, avec le numéro initial de B + 1.

La libération de la connexion se fait aussi par poignée de main, avec les segments **FIN**:



TCP se représente les données comme un flot continu d'octets indépendamment du nombre de segments nécessaires pour les transporter.

Chaque octet est numéroté séquentiellement à partir du numéro ISN (numéro initial de séquence), de sorte que le premier octet porte le numéro ISN+1. Le numéro inscrit dans un segment envoyé correspond au numéro du premier octet de données du segment.

TCP permet aussi le contrôle de flux par un mécanisme à fenêtre, car les segments comportent un champ fenêtre qui indique au destinataire le nombre d'octets qui peuvent être encore reçus. Ce champ indique à l'émetteur, qu'il peut poursuivre l'envoi de segments tant que le nombre total d'octets non-acquittés reste inférieur au nombre pouvant être reçus par le récepteur.

Les numéros de port (chacun sur 16 bits) identifient les processus émetteur et récepteur.

4. Définition d'une architecture client/serveur

Le modèle client-serveur :

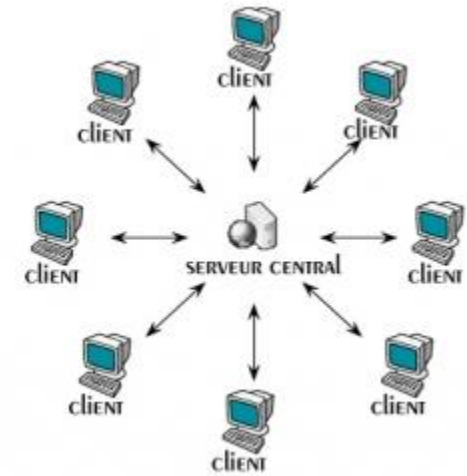
Une application fonctionne selon une architecture client-serveur quand :

➡ **les machines « clientes » contactent une machine « serveur » afin que ce serveur leur fournisse un service (via l'exécution d'un programme).**

- Les clients envoient des requêtes
- Le serveur envoie des réponses

C'est toujours le client qui déclenche la demande de service.
Le serveur attend passivement les requêtes des clients.

Un serveur traite plusieurs clients en même temps et contrôle leurs accès aux ressources. Le logiciel client-serveur masque aux clients la localisation du serveur. Le logiciel client-serveur est indépendant des plates-formes matérielles et logicielles.



Dans l'architecture **client-serveur**, une application est constituée de trois parties :

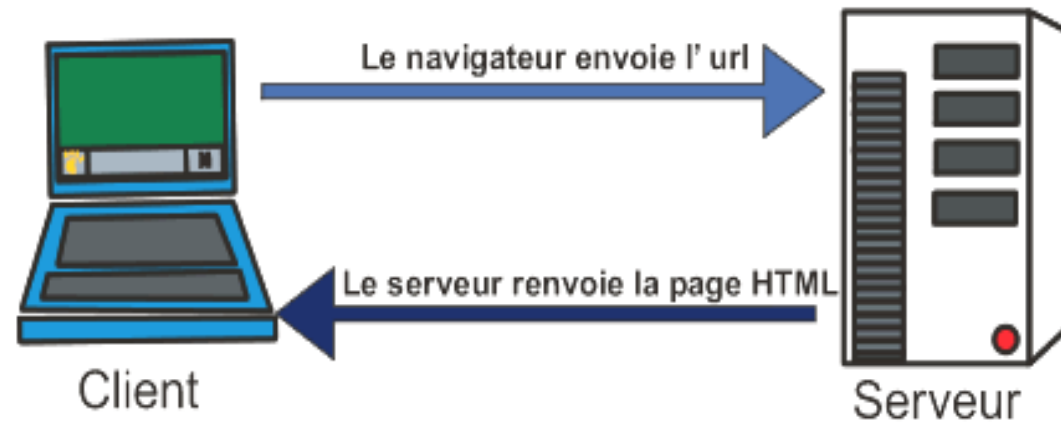
- l'interface utilisateur,
- la logique des traitements,
- la gestion des données.

Le client n'exécute que l'interface utilisateur (interfaces graphiques de type Windows) et la logique des traitements (formuler la requête), laissant au serveur de bases de données, la gestion complète des manipulations de données.

Le « - » qui apparaît dans client-serveur correspond à tout un ensemble complexe de logiciels appelé « **middleware** » qui se charge de toutes les communications entre les processus.

Dans Internet, la notion de **client-serveur** est sous-jacente. Mais réciproquement, on assiste aujourd'hui à une standardisation du client-serveur grâce à l'internet.

La notion du **client-serveur** s'applique totalement à l'internet. On parle **de service, de serveurs Web, de serveurs ftp ...et de client internet pour le navigateur.**



1) Envoie d'une requête



4) Réception des données et affichage

Le navigateur envoie l'URL (*Uniform Resource Locator*)



Le serveur répond en envoyant une page HTML (*HyperText Markup Language*)

2) Le serveur traite la requête



3) Le serveur envoie le résultat de la requête au client

Avantages et inconvénients Client-Serveur

Avantages :

- **Unicité de l'information** : toutes les données sont stockées sur un même serveur
- **Meilleure sécurité** : simplification des contrôles de sécurité item
- **Mise à jour** : mise à jour centralisé aussi bien des données et logiciels
- **Meilleure fiabilité** : En cas de panne, seul le serveur fait l'objet d'une réparation
- **Architecture plus mature que les autres**
- **Facilité d'évolution** : architecture évolutive, il est très facile de rajouter ou d'enlever des clients ou des serveurs.

Inconvénients :

- **Un coût d'exploitation élevé** (bande passante, câbles, ordinateurs surpuissants)
- **En cas de panne du serveur, plus aucun client n'a accès aux informations**
- **Si trop de clients** veulent communiquer avec le serveur ce dernier **risque de ne pas supporter la charge**